```
//
================================================================================
=======
// IMPLEMENTATION 1: SimpleHttpServer
//
================================================================================
=======


/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
package com.example.lab.based.practical.mock.http;


/*
 * ----------------
 * Please import necessary libraries
 * ----------------
*/
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpServer;
import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;
```

```
/*
 * ----------------
 * Please define a class named 'SimpleHttpServer' to encapsulate the HTTP server functionality.
 * ----------------
*/
public class SimpleHttpServer {

  /*
   * ----------------
   * Please define the main method, the entry point of the application.
   * This method sets up and starts the HTTP server.
   * ----------------
  */
  public static void main(String[] args) {

    /*
     * ----------------
     * Please create an HttpServer instance called server that listens on port 8080. Set the backlog to 0
     * Use try-with-resources to ensure the server is closed properly.
     * ----------------
    */
    try {
      HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);

      /*
```

```
         * ----------------
         * Create a context for "/myendpoint" and handle it with MyHandler.
         * ----------------
         */
        server.createContext("/myendpoint", new MyHandler());

        /*
         * ----------------
         * Set the server's executor to null (no thread pool).
         * ----------------
         */
        server.setExecutor(null);

        /*
         * ----------------
         * Start the server.
         * print "Server started on port 8080"
         * ----------------
         */
        server.start();
        System.out.println("Server started on port 8080");

    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

```
/*
 * ----------------
 * Define a static inner class 'MyHandler' to handle HTTP requests to '/myendpoint'.
 * This class implements HttpHandler
 * ----------------
*/
static class MyHandler implements HttpHandler {


    /*
     * -----------------------
     * Please override the handle method which accepts HttpExchange object called exchange which throws io exception
     * -----------------------
    */
    @Override
    public void handle(HttpExchange exchange) throws IOException {


        /*
         * ----------------
         * please get request method using exchange object and store it insdie a string variable called string
         * please get the request URI and the related path using exchange object and store it in a string variable called path
         * ----------------
        */
        String method = exchange.getRequestMethod();
```

```
String path = exchange.getRequestURI().getPath();

/*
 * ----------------
 * if the endpoint equals path
   * invoke/call the sendResponse method and pass necessary arguments.
   * then use return keyword
 * ----------------
*/
if (!"/myendpoint".equals(path)) {
   sendResponse(exchange, "Not Found", 404);
   return;
}

/*
 * ---------------
 * check if the request method is "GET"
 * ---------------
*/
if ("GET".equals(method)) {
   handleGetRequest(exchange);
}

/*
* ------------------
* otherwise check if the request method is "POST"
```

```
        * ------------------
        */
        else if ("POST".equals(method)) {

          /*
           * ------------------
           * define s atring varibale called response and store a message like "This is a POST request to /myendpoint"
           * call sendResponse method and provide necessary arguments
           * ------------------
           */
          String response = "This is a POST request to /myendpoint";
          sendResponse(exchange, response, 200);
        }

        /*
        * ----------------------
        * other wise Handle unsupported methods by calling sendResponse method and pass necessary arguments
        * ----------------------
        */
        else {
          sendResponse(exchange, "Method not supported", 405);
        }
      }

      /*
```

```
     * -------------------------
     * Please define a private void method called handleGetRequest which accepts an object of the HttpExhcnage called exchange,
       throwing io exception
     * -------------------------
    */
    private void handleGetRequest(HttpExchange exchange) throws IOException {

      /*
       * -------------
       * Please define a string variable called response and include this message: "This is a GET request to /myendpoint"
       * -------------
      */
      String response = "This is a GET request to /myendpoint";

      /*
       * -------------------
       * Please invoke sendResponse methos and pass necessary arguments
       * -------------------
      */
      sendResponse(exchange, response, 200);
    }

    /*
     * -------------------------
     * Please define a private void method called sendResponse and pass these arguments:
```

HttpExhange object: exchange, a string varibale reponse, an integer varibale statusCode

* -------------------------

*/

private void sendResponse(HttpExchange exchange, String response, int statusCode) throws IOException {

/*

* -------------

* Please get response headers using exchange object and Set the "Content-Type" header to "text/plain".

* -------------

*/

exchange.getResponseHeaders().set("Content-Type", "text/plain");

/*

* ------------------

* Please send response headers using exchange object and pass response code,

and convert the reponse to bytes and get the length of that

* ------------------

*/

exchange.sendResponseHeaders(statusCode, response.getBytes().length);

/*

* --------------------

* Please define a try block and inside it create an object of the OutputStream called os

```
            and get the response body using exchange object
         * Send the response using os object and pass reponse as its argument by converting it to bytes.
         * ----------
         */
        try (OutputStream os = exchange.getResponseBody()) {
            os.write(response.getBytes());
        }
      }
   }
}



//
================================================================================
=======
// IMPLEMENTATION 2: ClientServer
//
================================================================================
=======


/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package com.example.lab.based.practical.mock.solutions;
```

```
/*
 * ----------------
 * Please import libraries
 * ----------------
*/
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.logging.Level;
import java.util.logging.Logger;

/*
 * ----------------
 * Please define a class named 'ClientServer' that will contain both the server and client logic.
 * This demonstrates a simple client-server interaction within a single class.
 * ----------------
*/
public class ClientServer {

    /*
```

```
 * ----------------
 * Please create a logger and call it as logger
 * ----------------
*/
private static final Logger logger = Logger.getLogger(ClientServer.class.getName());

/*
 * ----------------
 * Please define a constant for the port number the server will listen on.
 * Using a constant makes it easy to change the port in one place.
 * ----------------
*/
private static final int PORT = 12345;

/*
 * ----------------
 * Please define the main method, the entry point of the application.
 * This method creates and starts the server and client threads.
 * ----------------
*/
public static void main(String[] args) {

   /*
    * ----------------
    * Please create a new thread for the server and start it.
    * The ServerSocket logic is encapsulated within the Server class.
```

```
        * ----------------
        */
        Thread serverThread = new Thread(new Server());
        serverThread.start();

        /*
         * ----------------
         * Please create a new thread for the client and start it.
         * The client logic is encapsulated within the Client class.
         * ----------------
        */
        Thread clientThread = new Thread(new Client());
        clientThread.start();
    }

    /*
     * ----------------
     * Please define a nested class named 'Server' that implements the Runnable interface.
     * This class encapsulates the server's functionality and allows it to run in a separate thread.
     * ----------------
    */
    static class Server implements Runnable {

        /*
         * ----------------
```

```
     * Please override the run method from the Runnable interface.
     * This method contains the main logic for the server.
     * ----------------
     */
    @Override
    public void run() {

        /*
         * ----------------
         * Please create a ServerSocket to listen for incoming client connections.
         * Use try-with-resources to ensure the ServerSocket is closed properly.
         * Inside the try block please please use informational logger to log "Server started on port 12345"
         * Handle potential IOExceptions during ServerSocket creation.
         * ----------------
         */
        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            logger.info("Server started on port " + PORT);

            /*
             * ----------------
             * Please loop indefinitely, listening for and accepting client connections.
             * ----------------
             */
            while (true) {
```

```
            /*
             * ----------------
             * Please accept an incoming client connection uaing serverSocket object and store it inside an object of the Socket class called clientSocket.
             * Please print "Client connected" plus the clientSocket object
             * ----------------
            */
            Socket clientSocket = serverSocket.accept();
            System.out.println("Client connected " + clientSocket);

            /*
             * ----------------
             * Please create a new ClientHandler thread to handle communication with the newly connected client.
             * Pass the client's socket to the ClientHandler. Start the handler thread.
             * ----------------
            */
            Thread handlerThread = new Thread(new ClientHandler(clientSocket));
            handlerThread.start();
        }
    } catch (IOException e) {
        logger.log(Level.SEVERE, "Server error", e);
    }
}

/*
 * ----------------
```

```
     * Please define a nested class named 'ClientHandler' that implements the Runnable interface.
     * This class handles the interaction with a single connected client.
     * ----------------
     */
    static class ClientHandler implements Runnable {

        /*
         * ----------------
         * Please define a constructor for the ClientHandler class.
         * This constructor takes the client's socket as a parameter.
         * ----------------
         */
        private final Socket clientSocket;

        public ClientHandler(Socket clientSocket) {
            this.clientSocket = clientSocket;
        }

        /*
         * ----------------
         * Please override the run method of the Runnable interface.
         * This method contains the logic for communicating with the client.
         * ----------------
         */
        @Override
```

```
    public void run() {


        /*
         * ----------------
         * Use try-with-resources to ensure resources are closed automatically.
         * Handle potential IOExceptions.
         * ----------------
        */
        try {


            /*
             * ----------------
             * Please create a BufferedReader to read text data from the client socket's input stream.
             * BufferedReader buffers the input for efficient reading of characters, lines, and arrays.
             * Wrap the getInputStream() (which provides raw byte stream) with an InputStreamReader
             * to convert the byte stream into a character stream.
             * ----------------
            */
            BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));


            /*
             * ----------------
```

```
            * Please create a PrintWriter to send formatted text data to the client socket's
output stream.
            * PrintWriter provides convenient methods for writing various data types
(strings, numbers, etc.) as text.
            * The 'true' argument enables auto-flushing, ensuring that data is sent
immediately after each print/println call,
            * preventing buffering-related delays.  Wrap getOutputStream() (which provides
raw byte stream).
            * ----------------
            */
            PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);


          /*
           * ----------------
           * Please read messages from the client in a loop and store it inside a string
varibale called inputLine.
           * Continue reading until the client sends "exit" or the connection is closed.
           * print the inputLine plus a message like "Server received: "
           * ----------------
          */
          String inputLine;
          while ((inputLine = in.readLine()) != null) {
            System.out.println("Server received: " + inputLine);


            /*
             * --------------
             * Please check if the client has sent the "exit" command.
```

```
                 * --------------
                */
                if ("exit".equals(inputLine)) {


                   /*
                    * --------------
                    * If "exit" is received, send a "Bye" message to the client and break out of the loop.
                    * The println() method of PrintWriter writes a string followed by a newline character to the output stream.
                    * Then break the operation
                    * otherwise
                    * --------------
                   */
                   out.println("Bye");
                   break;
                } else {


                   /*
                    * --------------
                    * send the inputLine plus "Echo: "
                    *
                    * --------------
                   */
                   out.println("Echo: " + inputLine);
                }
             }
```

```
            } catch (IOException e) {
                logger.log(Level.SEVERE, "ClientHandler error", e);
            }

            /*
             * --------------
             * create a try block and close the clientSocket and catch io exception
             * --------------
            */
            try {
                clientSocket.close();
            } catch (IOException e) {
                logger.log(Level.SEVERE, "Error closing client socket", e);
            }
        }
    }
}

/*
 * ----------------
 * Please define a nested class named 'Client' that implements the Runnable interface.
 * This class encapsulates the client's functionality and allows it to run in a separate thread.
 * ----------------
*/
```

```
static class Client implements Runnable {

    /*
     * ----------------
     * Please override the run method from the Runnable interface.
     * This method contains the main logic for the client.
     * ----------------
    */
    @Override
    public void run() {

        /*
         * ----------------
         * Use try-with-resources to ensure resources are closed automatically.
         * ----------------
        */
        try {

            /*
             * --------------------
             * Please create a Socket object, establishing a connection to the server at the specified host
              ("localhost") and port (PORT).
             * This initiates the connection to the server's ServerSocket.
             * --------------------
            */
```

```
        Socket socket = new Socket("localhost", PORT);


        /*
         * ----------------
         * Please create a PrintWriter to send formatted text data to the server through the socket's output stream.
         * PrintWriter simplifies sending text-based data.
         * 'true' for auto-flush: ensures data is sent immediately without buffering delays.
         * Wrap socket.getOutputStream() to provide character-based output.
         * ----------------
        */
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);


        /*
         * ----------------
         * Please Create a BufferedReader to read text data from the server's responses through the socket's input stream.
         * BufferedReader provides efficient reading of characters, lines, and arrays by buffering the input.
         * Wrap socket.getInputStream() with InputStreamReader to convert bytes to characters.
         * ----------------
        */
        BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));


        /*
```

```
            * ------------------
            * Please create a BufferedReader to read text input from the console (System.in).
            * This allows the client to read user input.
            * Wrap System.in (which is an InputStream) with an InputStreamReader to handle character encoding.
            * ------------------
            */
            BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in));

        /*
         * ----------------
         * Please read input from the console  in a loop.
         * ----------------
        */
        String userInput;
        while ((userInput = stdIn.readLine()) != null) {

            /*
             * ----------------
             * Please send the user's input to the server using the PrintWriter.
             * ----------------
            */
            out.println(userInput);

            /*
             * ----------------
```

```
         * Please read the server's response using the BufferedReader and store it inside a string variable called receivedMessage .
         * check if the receivededMessage is null
         * Then print "Server closed the connection."
         * exit the loop
         * ----------------
        */
        String receivedMessage = in.readLine();
        if (receivedMessage == null) {
           System.out.println("Server closed the connection.");
           break;
        }

        /*
         * ---------------
         * Print Server Response
         * ---------------
        */
        System.out.println("Server response: " + receivedMessage);

        /*
         * ----------------
         * Please check if the user typed "exit". If so, exit the loop.
         * ----------------
        */
        if ("exit".equals(userInput)) {
```

```
                break;
            }
          }
        } catch (IOException e) {
          logger.log(Level.SEVERE, "Client error", e);
        }
      }
    }
}
```